

New Input By Reference

Documentation

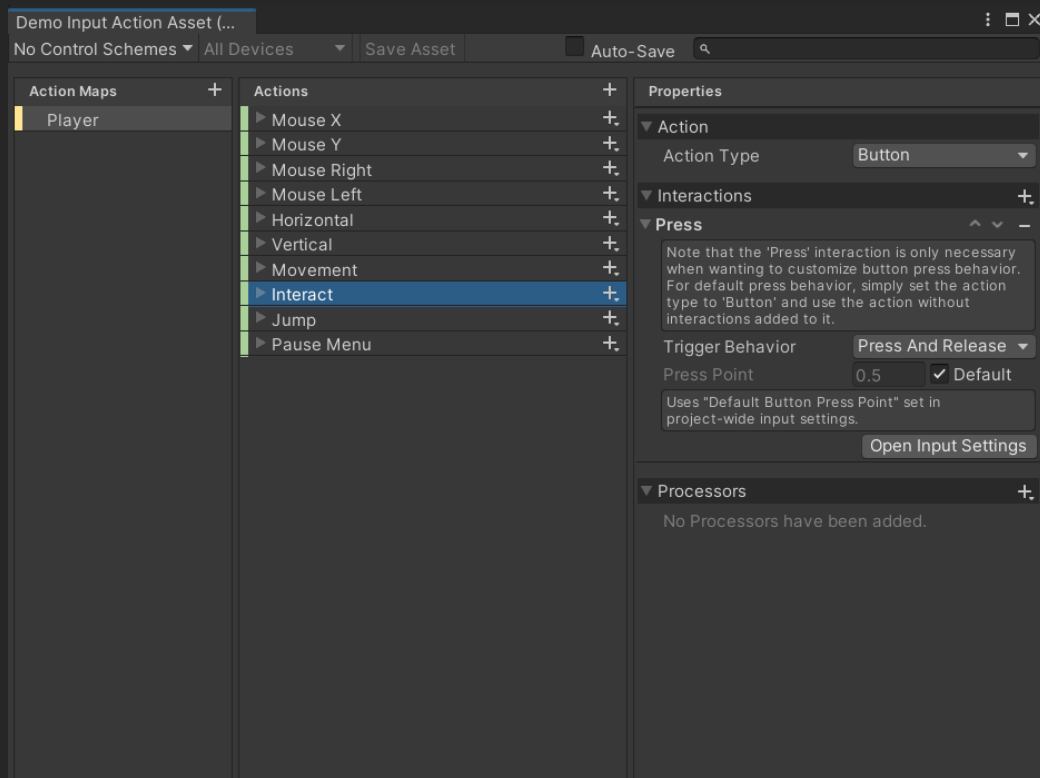
by The BlindEye

Contents

1. What is included with the Package?
2. What are the Functions?
3. How to set up a Project to use the Package?
4. What do custom errors indicate?
5. What are some script examples?
6. What are some tips?

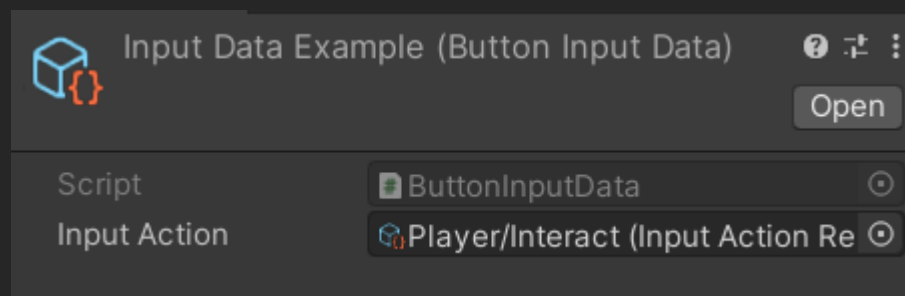
1. What is included with the Package?

- **Demo Input Action Asset** - it is an Input Action Asset that contains a collection of Input Actions that can be extended upon.

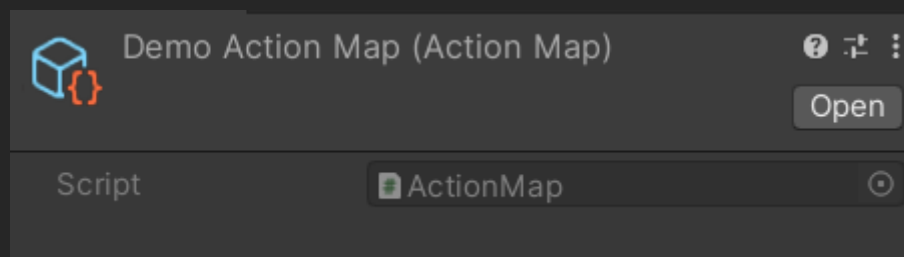


Remark: All **Button Action Types** require a **Press Interaction** with the **Trigger Behavior** set to **Press and Release** in order for the `NewInput.GetButtonDown` function to behave properly (as shown in the image above).

- **Demo Input Handler** - it is a Prefab that's already configured to function with the Demo Input Action Asset and can be used to skip the steps without * from point 3 of the Contents.
- **Input Data** - it is a Scriptable Object that stores a reference to an Input Action.



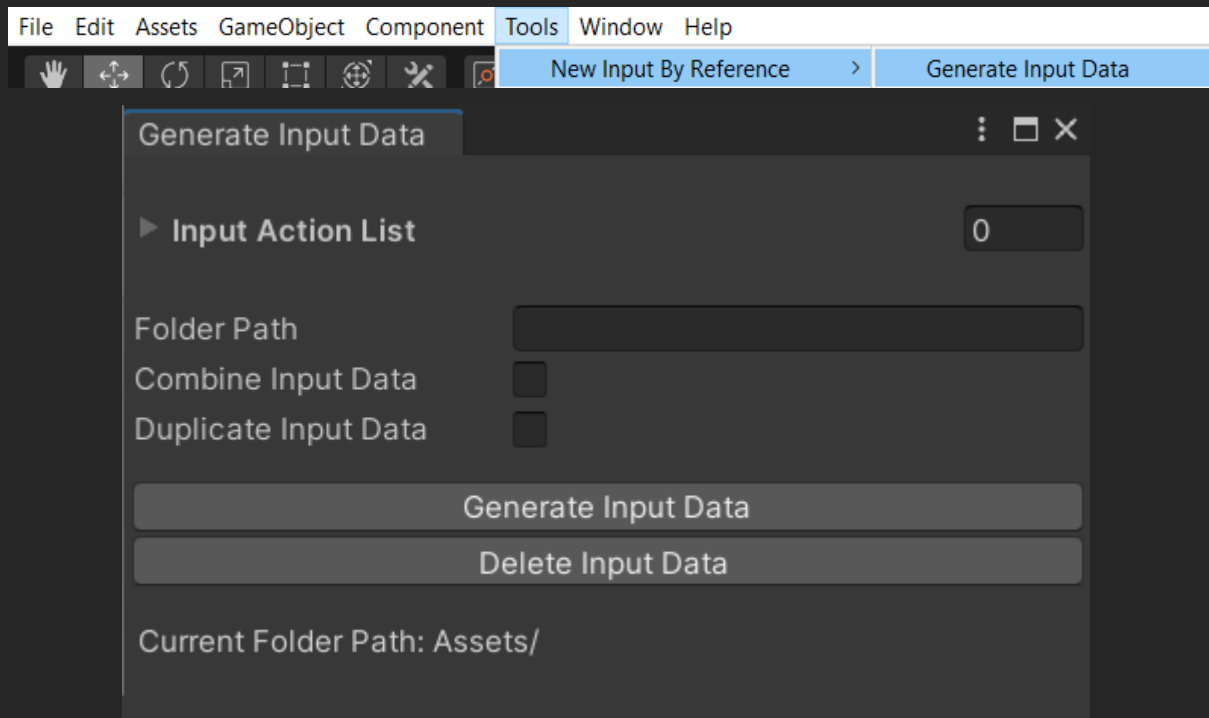
- **Action Map** - it is a Scriptable Object that stores a List of the Input Data inside it, that can be accessed by using **ActionMap.InputDataList**. It can be only generated using Generate Input Data Window.



Remark: A **Demo Action Map** is included in the Package, which was generated from the **Demo Input Action Asset**.

- **Bindings Images** - it is a Scriptable Object that stores references to the Sprites assigned into the **Generate Bindings Images Window**, for keyboard, gamepad and mouse keys.

- **Generate Input Data Window** - it is a Custom Editor Window that takes a list of Input Actions and generates Input Data from it.



Folder Path: Provide a Folder Path. Default Folder Path is *Assets/*

Combine Input Data: Combine generated Input Data into a single Action Map

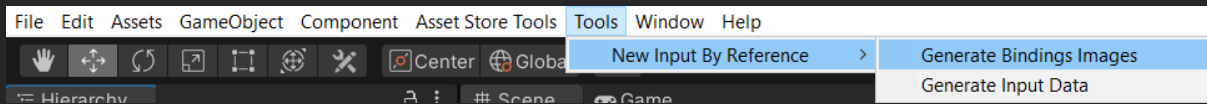
Duplicate Input Data: Generate Input Data even if in the Current Folder Path is already a duplicate of it

Current Folder Path: The location where the Input Data will be generated/deleted

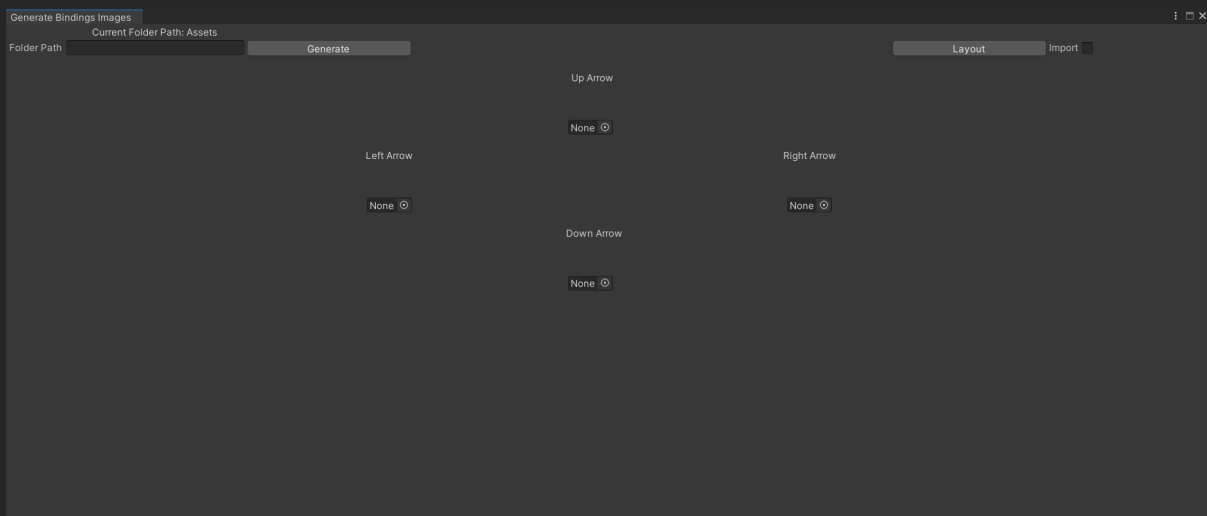
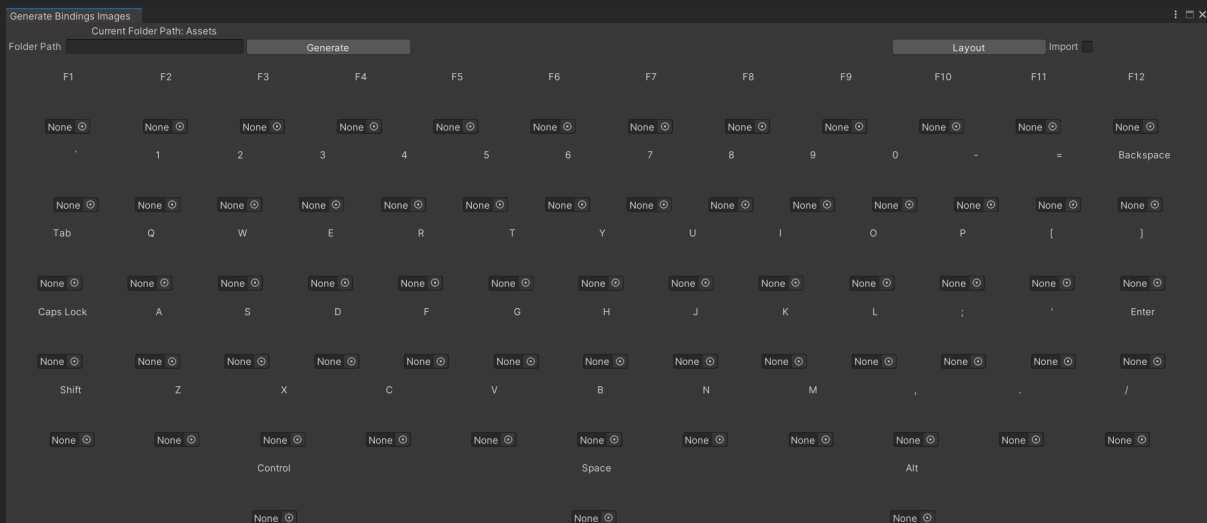
Generate Input Data: Generate Input Data to the Current Folder Path

Delete Input Data: Delete Input Data from the Current Folder Path

- **Generate Bindings Images Window** - it is a Custom Editor Window where Sprites can be assigned for keyboard, gamepad and mouse keys, and then accessed thru code with `NewInput.GetBindingSprite(InputData inputData, int bindingIndex = 0)`



Keyboard Keys



Mouse Keys

Generate Bindings Images

Current Folder Path: Assets

Generate

Layout

Import

Folder Path

LMB

MMB

RMB

None

None

None

Forward

None

Back

None

Legend:

LMB = Left Mouse Button

MMB = Middle Mouse Button

RMB = Right Mouse Button

Gamepad Keys

Generate Bindings Images

Current Folder Path: Assets

Generate

Layout

Import

LB

LT

RT

RB

None

None

None

None

LS/Up

Y

None

LS/Left

Left Stick Press

LS/Right

Select

Start

X

B

None

None

None

None

D-Pad/Up

LS/Down

A

RS/Up

None

None

D-Pad/Left

D-Pad/Right

RS/Left

Right Stick Press

RS/Right

None

None

None

D-Pad/Down

D-Pad

LS

RS

RS/Down

None

None

None

None

Folder Path: Provide a Folder Path. Default Folder Path is *Assets/*

Current Folder Path: The location where the BindingsImages Scriptable Object will be generated

Generate: Generate the BindingsImages Scriptable Object to the Current Folder Path

Layout: Change the current layout

Import (CheckBox): Import a generated BindingsImages Scriptable Object

Import (Button): Import the selected BindingsImages Scriptable Object

Save: Save the changes made to the selected BindingsImages Scriptable Object

2. What are the Functions?

- To access the functions, **using NewInputByReference;** is needed at the top of the script.

Action Map Functions

```
void NewInput.SwitchActionMap(string actionMap)
```

Summary	Change the current Action Map to the one indicated by "actionMap".
actionMap	Can be found in the Input Action Asset, under the "Action Maps" column.

```
void NewInput.EnableActionMap(string actionMap)
```

Summary	Enable the Action Map indicated by "actionMap". Other Action Maps will not be disabled if they are currently active.
actionMap	Can be found in the Input Action Asset, under the "Action Maps" column.

```
void NewInput.DisableActionMap(string actionMap)
```

Summary	Disable the Action Map indicated by "actionMap". Other Action Maps will not be disabled if they are currently active.
actionMap	Can be found in the Input Action Asset, under the "Action Maps" column.

Rebind Functions

```
void NewInput.StartRebinding(  
    InputData inputData,  
    int bindingIndex = 0,  
    bool allCompositeBindings = false,  
    Action onComplete = null,  
    InputData cancelInputData = null,  
    InputData excludedInputData = null)
```

Summary	Start the rebinding process for the input action indicated by "inputData", by waiting for a key input from the player.
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference.
bindingIndex	Index of the input action, indicated by "inputData", binding that will be rebounded. If "isComposite" is true, "bindingIndex" becomes the starting index of the composite. Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6. What are some tips?", to understand how they are numbered)
allCompositeBindings	Can be true only if the input action indicated by "inputData" is of Action Type = Value and Control Type = Axis/Vector 2/Vector 3. If it's true, all the input actions from the same composite, as the input action indicated by "inputData", will start the rebinding process one after another. After all of them have been rebounded, the function will exit. (check Documentation -> "6. What are some tips?", to understand what is a composite)
onComplete	Delegate that invokes when the rebinding process is completed.
cancelInputData	Input action indicated by "cancelInputData" that upon pressing makes the function exit, and stops listening to the player's input. (Default Action Path = <Keyboard>/escape) (cancelInputData.InputAction.bindings.Count <= 2)
excludedInputData	Input action indicated by "excludedInputData" that upon press will not rebind(nor exit the function) the input action indicated by "inputData". (excludedInputData.InputAction.bindings.Count <= 2)

```
void NewInput.Rebind(InputData inputData, string
bindingPath, int bindingIndex = 0)
```

Summary	Rebind the input action indicated by "inputData".
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference.
bindingPath	Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Path. (e.g. <Keyboard>/e, <Gamepad>/leftStick) Remark: To avoid GC, the passed variable should be cached into the class.
bindingIndex	Index of the input action, indicated by "inputData", binding that will be rebounded. Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6.What are some tips?", to understand how they are numbered)

```
void NewInput.Rebind(string actionName, string
bindingPath, int bindingIndex = 0)
```

Summary	Rebind the input action indicated by "actionName".
actionName	Can be found in the Input Action Asset, under the "Actions" column.
bindingPath	Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Path. (e.g. <Keyboard>/e, <Gamepad>/leftStick) Remark: To avoid GC, the passed variable should be cached into the class.
bindingIndex	Index of the input action, indicated by "actionName", binding that will be rebounded. Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6.What are some tips?", to understand how they are numbered)

```

    string NewInput.GetBindingName(
        InputData inputData,
        int bindingIndex = 0,
        bool allCompositeBindings = false,
        NameOptions nameOption = NameOptions.OmitDevice)

```

Summary	Get the name of the current binding indicated by "inputData". (e.g. W, E, Up Arrow, Tab, etc.)
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference.
bindingIndex	Index of the input action, indicated by "inputData", binding that will provide the name. If "bindingIndex" = 0 and the input action, indicated by "actionName", is a composite, the function will return all the bindings' names separated by "/". (e.g. W/S/A/D, S/W, A/D) Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6.What are some tips?", to understand how they are numbered)
allCompositeBindings	Can be true only if the input action indicated by "inputData" is of Action Type = Value and Control Type = Axis/Vector 2/Vector 3. If it's true, all the input actions from the same composite as the input action, indicated by "inputData", will return their bindings' names separated by "/" (e.g. W/S/A/D, S/W, A/D), and "bindingIndex" will become the starting index of the composite (check Documentation -> "6.What are some tips?", to understand what is a composite and its starting index)
nameOption	Options for customizing the behavior of the returned string.

```

        string NewInput.GetBindingName(
string actionName,
int bindingIndex = 0,
bool allCompositeBindings = false,
NameOptions nameOption = NameOptions.OmitDevice)

```

Summary	Get the name of the current binding indicated by "inputData". (e.g. W, E, Up Arrow, Tab, etc.)
actionName	Can be found in the Input Action Asset, under the "Actions" column.
bindingIndex	Index of the input action, indicated by "actionName", binding that will provide the name. If "bindingIndex" = 0 and the input action, indicated by "actionName", is a composite, the function will return all the bindings' names separated by "/". (e.g. W/S/A/D, S/W, A/D) Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6.What are some tips?", to understand how they are numbered)
allCompositeBindings	Can be true only if the input action indicated by "actionName" is of Action Type = Value and Control Type = Axis/Vector 2/Vector 3. If it's true, all the input actions from the same composite as the input action, indicated by "actionName", will return their bindings' names separated by "/" (e.g. W/S/A/D, S/W, A/D), and "bindingIndex" will become the starting index of the composite (check Documentation -> "6.What are some tips?", to understand what is a composite and its starting index)
nameOption	Options for customizing the behavior of the returned string.

```
Sprite GetBindingSprite(InputData inputData, int
bindingIndex = 0)
```

Summary	Get the sprite of the binding indicated by "inputData". A BindingsImages Scriptable Object needs to be assigned using NewInput.SetBindingsImages function, or in the bindingsImages field of the InputHandler.
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference.
bindingIndex	Index of the input action, indicated by "inputData", binding that will provide the sprite. Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6.What are some tips?", to understand how they are numbered)

```
Sprite GetBindingSprite(string actionName, int
bindingIndex = 0)
```

Summary	Get the name of the current binding indicated by "inputData". (e.g. W, E, Up Arrow, Tab, etc.)
actionName	Can be found in the Input Action Asset, under the "Actions" column.
bindingIndex	Index of the input action, indicated by "actionName", binding that will provide the sprite. If "bindingIndex" = 0 and the input action, indicated by "actionName", is a composite, the function will return all the bindings' names separated by "/". (e.g. W/S/A/D, S/W, A/D) Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6.What are some tips?", to understand how they are numbered)

```
void NewInput.ResetRebinds()
```

Summary	Reset all rebinds of the current Input Action Asset, attached to the Player Input Component, to the default value.
---------	--

```
void NewInput.ResetRebind(InputData inputData,  
int bindingIndex = 0)
```

Summary	Reset the rebinding of the input action indicated by "inputData".
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference.
bindingIndex	Index of the input action, indicated by "inputData", binding that will be reset. If "bindingIndex" = 0, the function will reset all rebinds of the input action, indicated by "inputData". Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6.What are some tips?", to understand how they are numbered)

```
void NewInput.ResetRebind(string actionName, int  
bindingIndex = 0)
```

Summary	Reset the rebinding of the input action indicated by "actionName".
actionName	Can be found in the Input Action Asset, under the "Actions" column.
bindingIndex	Index of the input action, indicated by "actionName", binding that will be reset. If "bindingIndex" = 0, the function will reset all rebinds of the input action, indicated by "actionName". Can be determined in the Input Action Asset, under the "Actions" column -> (any) Action -> Left DropDown Arrow. (check Documentation -> "6.What are some tips?", to understand how they are numbered)

Button Action Type Functions

```
bool NewInput.GetButtonDown(string actionName)
```

Summary	Returns true if the user pressed the virtual Button indicated by "actionName" during the current frame.
actionName	Can be found in the Input Action Asset, under the "Actions" column. (Action Type = Button)

```
bool NewInput.GetButtonDown(InputData inputData)
```

Summary	Returns true if the user pressed the virtual Button indicated by "inputData" during the current frame.
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference/Button Input Data.

```
bool NewInput.GetButton(string actionName)
```

Summary	Returns true if the user had pressed and did not release the virtual Button indicated by "actionName" during the current frame.
actionName	Can be found in the Input Action Asset, under the "Actions" column. (Action Type = Button)

```
bool NewInput.GetButton(InputData inputData)
```

Summary	Returns true if the user had pressed and did not release the virtual Button indicated by "inputData" during the current frame.
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference/Button Input Data.

```
bool NewInput.GetButtonUp(string actionName)
```

Summary	Returns true if the user released the virtual Button indicated by "actionName" during the current frame.
actionName	Can be found in the Input Action Asset, under the "Actions" column. (Action Type = Button)

```
bool NewInput.GetButtonUp(InputData inputData)
```

Summary	Returns true if the user released the virtual Button indicated by "inputData" during the current frame.
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference/Button Input Data.

Value Action Type Functions

```
float NewInput.GetAxis(string actionName)
```

Summary	Returns the value of the virtual Axis indicated by "actionName".
actionName	Can be found in the Input Action Asset, under the "Actions" column. (Action Type = Value, Control Type = Axis)

```
float NewInput.GetAxis(InputData inputData)
```

Summary	Returns the value of the virtual Axis indicated by "inputData".
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference/Axis Input Data.

```
Vector2 NewInput.GetVector2(string actionName)
```

Summary	Returns the value of the virtual Vector2 indicated by "actionName".
actionName	Can be found in the Input Action Asset, under the "Actions" column. (Action Type = Value, Control Type = Vector 2)

```
Vector2 NewInput.GetVector2(InputData inputData)
```

Summary	Returns the value of the virtual Vector2 indicated by "inputData".
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference/Vector2 Input Data.

```
Vector3 NewInput.GetVector3(string actionName)
```

Summary	Returns the value of the virtual Vector3 indicated by "actionName".
actionName	Can be found in the Input Action Asset, under the "Actions" column. (Action Type = Value, Control Type = Vector 3)

```
Vector3 NewInput.GetVector3(InputData inputData)
```

Summary	Returns the value of the virtual Vector3 indicated by "inputData".
inputData	Can be generated using Generate Input Data Window, or created from Create/New Input By Reference/Vector3 Input Data.

- There are also shortcuts for the functions above when using Input Data references.

Button Action Type

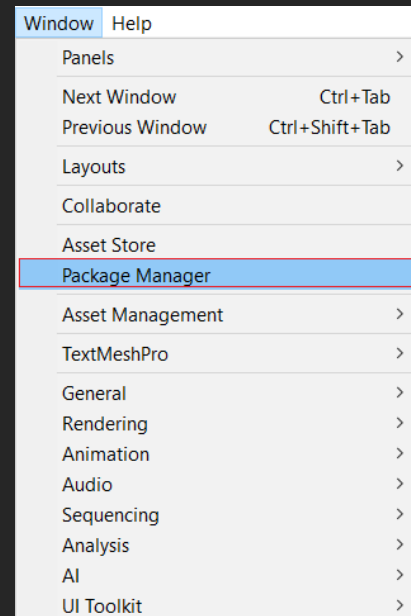
```
bool ButtonInputData.ButtonPressed  
bool ButtonInputData.ButtonHolding  
bool ButtonInputData.ButtonReleased
```

Value Action Type

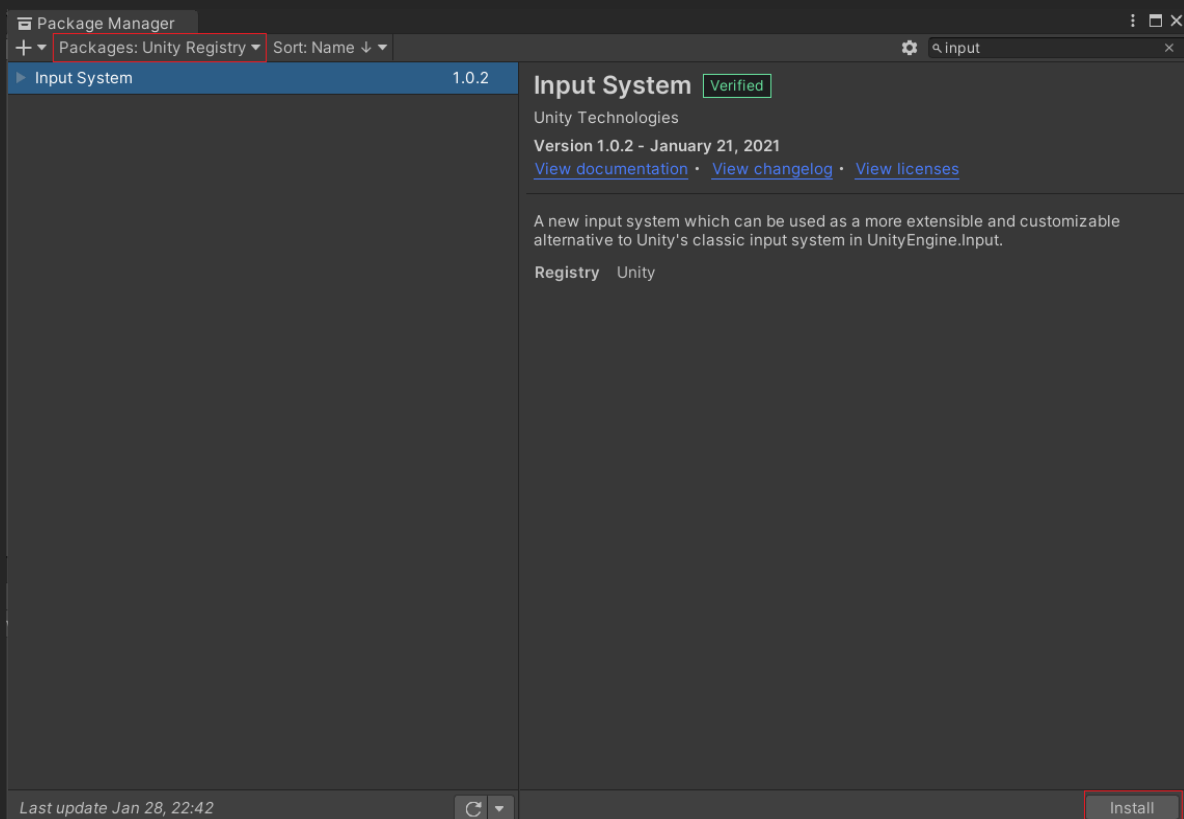
```
float AxisInputData.Axis  
Vector2 Vector2InputData.Vector2  
Vector3 Vector3InputData.Vector3
```

3. How to set up a Project to use the Package?

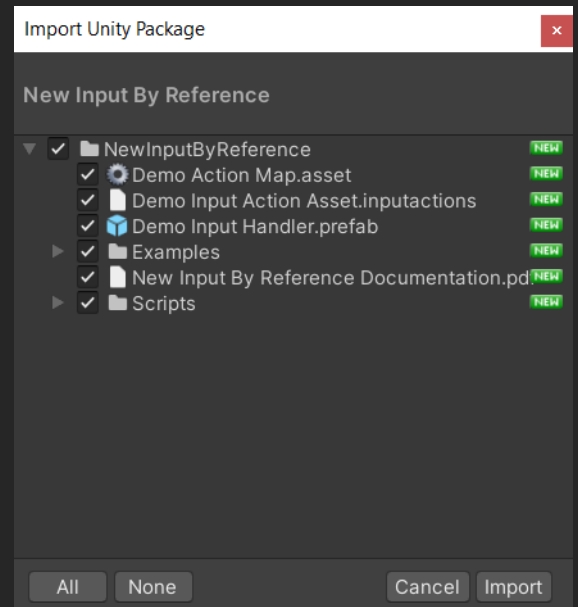
- *Open the **Package Manager Window** from Window/Package Manager



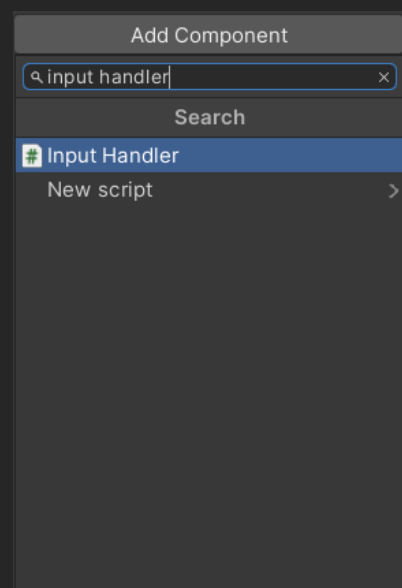
- *Install the **Input System Package** from the **Unity Registry**



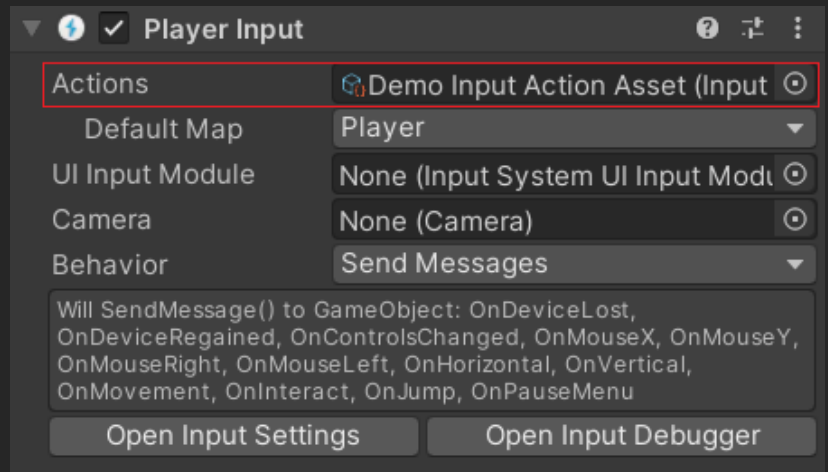
- *Import the **New Input By Reference Package** into the project.



- There must be an **Input Handler** script attached to a game object in the scene (frequently found on the player).



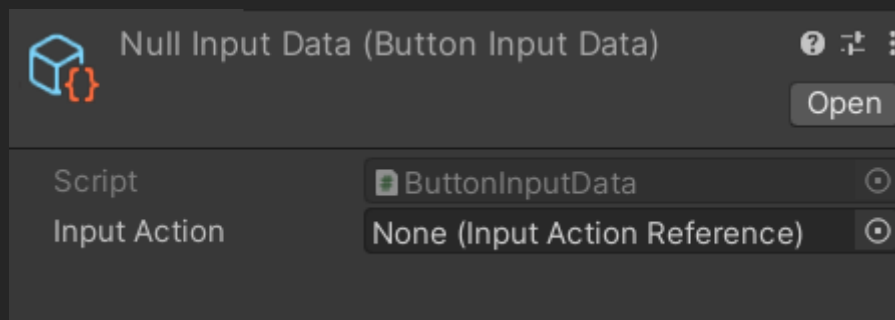
- On the **Player Input Component** which was added by the Input Handler script, an **Input Action Asset** needs to be assigned to the **Actions** field (the package comes with a demo version of an Input Action Asset that can be used out of the box).



4. What do custom errors indicate?

- Error01: Input Action is **Null**. Ensure that an Input Action is assigned to all Input Data, or if there is an Input Handler Script and a Player Input Script attached to the same game object in the scene.

The error indicates that an **Input Action** field from an **Input Data** is Null.



To solve this error, we need to assign an **Input Action** to the **Input Data**.

- Error02: Invalid Control Type for **Action Name**. Got **Type**, expected **Type**. Action Map: **Action Map Name**.

The error indicates that a **wrong parameter** was passed to a *NewInput*.function.

In this example, we are passing a **Vector2InputData** as a parameter for the **NewInput.GetButtonDown** (a function that can only take **ButtonInputData** or **Button Type Action Name**).

```
[SerializeField] private Vector2InputData inputData;
```

🔥 Event function

```
private void Update()
{
    if(NewInput.GetButtonDown(inputData))
        Debug.Log(message: "GetButtonDown Function");
}
```

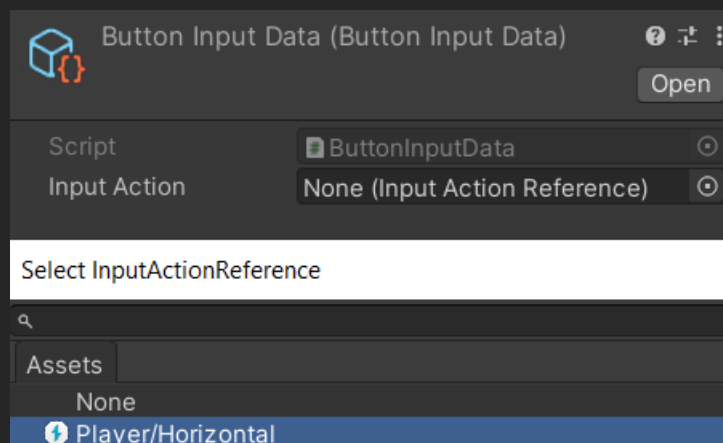
We're getting the following error: Invalid Control Type for **Action Movement**. Got **Vector2**, expected **Single**. Action Map: **Player Controls**.

To solve the error, we need to modify the **inputData** field's type to **ButtonInputData**.

- Error06: Invalid Control Type for **Action Name**.

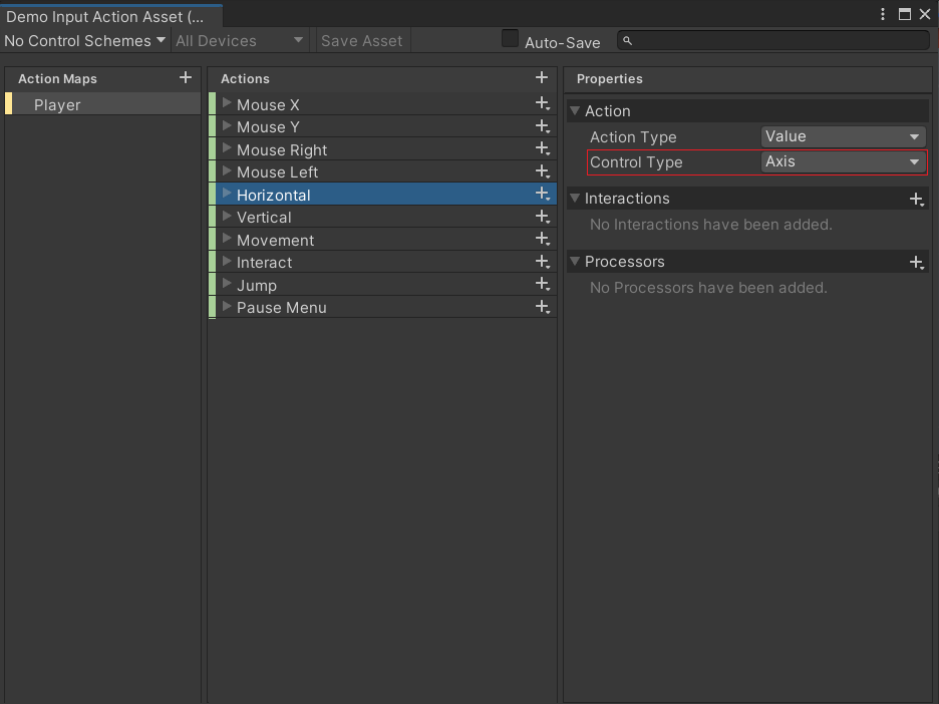
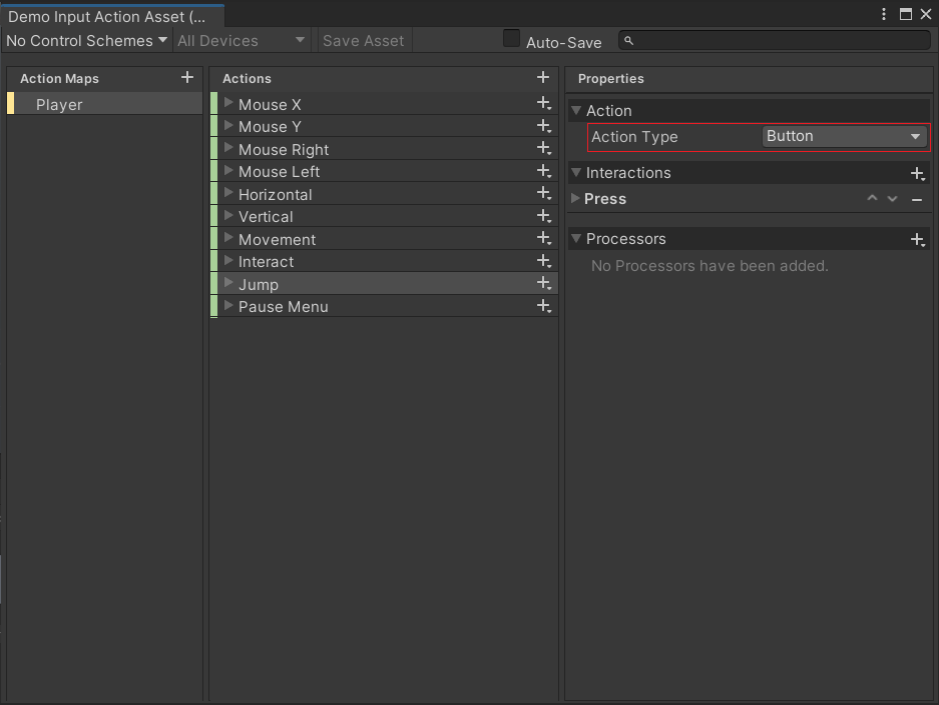
The error indicates a **wrong Input Action Control Type** assigned to the **Input Data**.

In this example, we are trying to assign **Horizontal Action** to a **Button Input Data**. As a result, we are getting the following error: Invalid Control Type for **Action Horizontal**.



To solve the error, we need to assign a **Button Control Type Action** to the field.

Remark: The **Control Type** field can be found in the **Properties Column** of the **Input Action Asset**. Keep in mind that while Unity does not display a **Control Type** field for **Button Action Types**, it assigns it to **Button** behind the scenes.



5. What are some script examples?

- All the scripts below can be found in the **NewInputByReference/Examples** folder.

```
public class MouseLook : MonoBehaviour
{
    [SerializeField] private float mouseSensitivity = 100f;  ⓘ "50"
    [SerializeField] private Transform playerBody;  ⓘ Player (Transform)

    private float _xRotation;

    ⓘ Event function
    private void Start () => Cursor.lockState = CursorLockMode.Locked;

    ⓘ Event function
    private void Update ()
    {
        float mouseX = NewInput.GetAxis( actionName: "Mouse X") * mouseSensitivity * Time.deltaTime;
        float mouseY = NewInput.GetAxis( actionName: "Mouse Y") * mouseSensitivity * Time.deltaTime;

        _xRotation -= mouseY;
        _xRotation = Mathf.Clamp( value: _xRotation, min: -90f, max: 90f);

        transform.localRotation = Quaternion.Euler(_xRotation, y: 0f, z: 0f);
        playerBody.Rotate( eulers: Vector3.up * mouseX);
    }
}
```



```

public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private float moveSpeed;  ⚙️ "15"
    [SerializeField] private float jumpHeight = 3f;  ⚙️ Unchanged
    [SerializeField] private float gravity = -9.81f;  ⚙️ Unchanged

    [SerializeField] private Transform groundCheck;  ⚙️ Ground Check (Transform)
    [SerializeField] private LayerMask groundMask;  ⚙️ Serializable
    [SerializeField] private float groundDistance = 0.4f;  ⚙️ Unchanged

    private CharacterController _controller;

    private Transform _transform;
    private Vector3 _velocity;
    private bool _isGrounded;

    ⚙️ Event function
    private void Awake()
    {
        _controller = GetComponent<CharacterController>();
        _transform = transform;
    }

    ⚙️ Event function
    private void Update()
    {
        _isGrounded = Physics.CheckSphere(groundCheck.position, groundDistance, (int) groundMask);
        if (_isGrounded && _velocity.y < 0)
            _velocity.y = -2f;

        float x = NewInput.GetAxis( actionName: "Horizontal");
        float z = NewInput.GetAxis( actionName: "Vertical");

        if(NewInput.GetButtonDown( actionName: "Jump") && _isGrounded)
            _velocity.y = Mathf.Sqrt( 6 * jumpHeight * -2f * gravity);

        _velocity.y += gravity * Time.deltaTime;

        var move :Vector3 = _transform.right * x + _transform.forward * z;
        move = Vector3.ClampMagnitude(move, maxLength: 1);

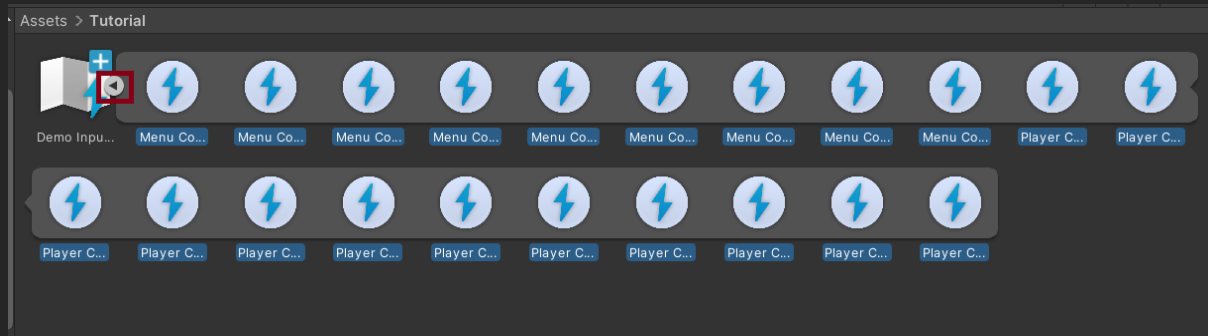
        _controller.Move( motion: (_velocity * Time.deltaTime) + (move * moveSpeed * Time.deltaTime));
    }
}

```

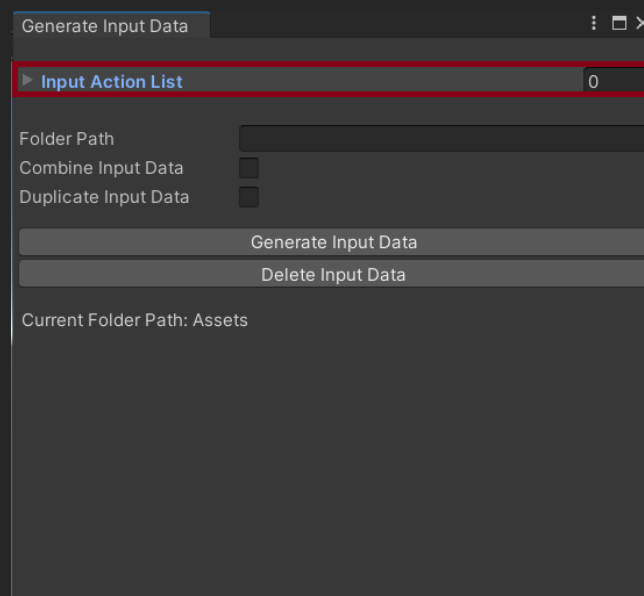
6. What are some tips?

- How can the Input Action List of the Generate Input Data Window be assigned more quickly?

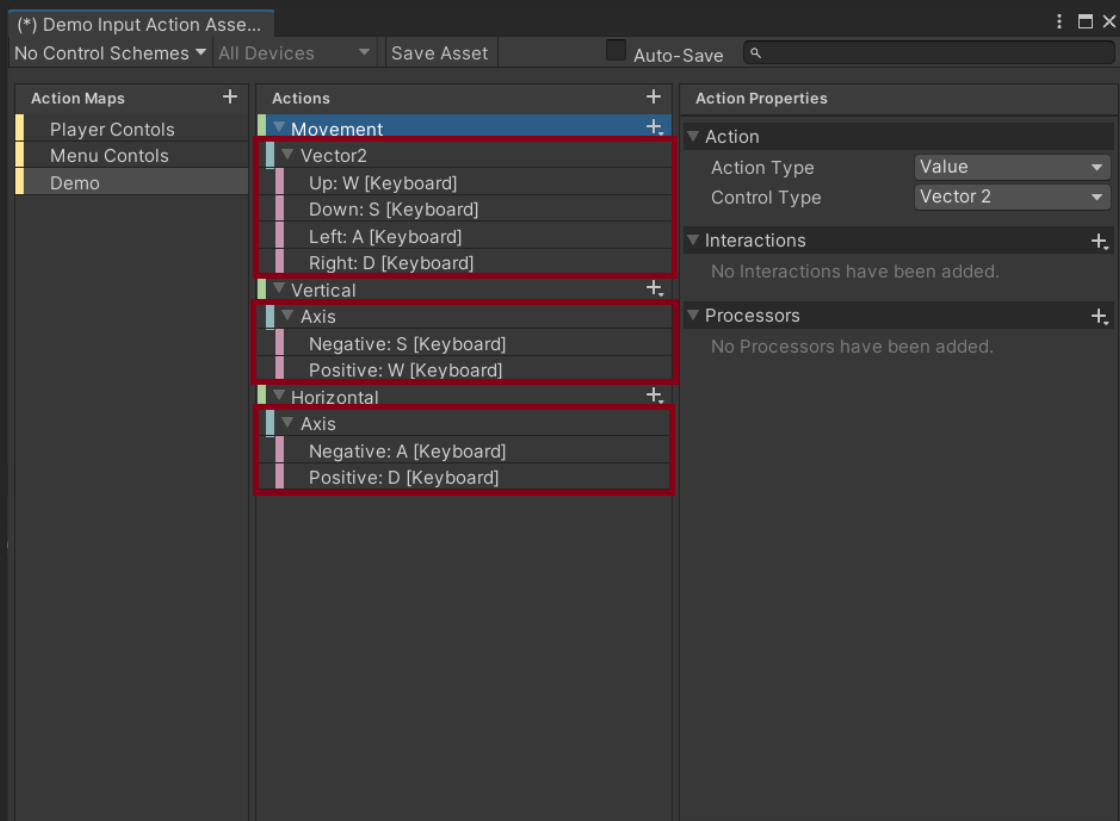
Expand the Input Action Asset and select all the Input Actions (Shift + Click).



Drag them onto the Input Action List from the Generate Input Data Window.

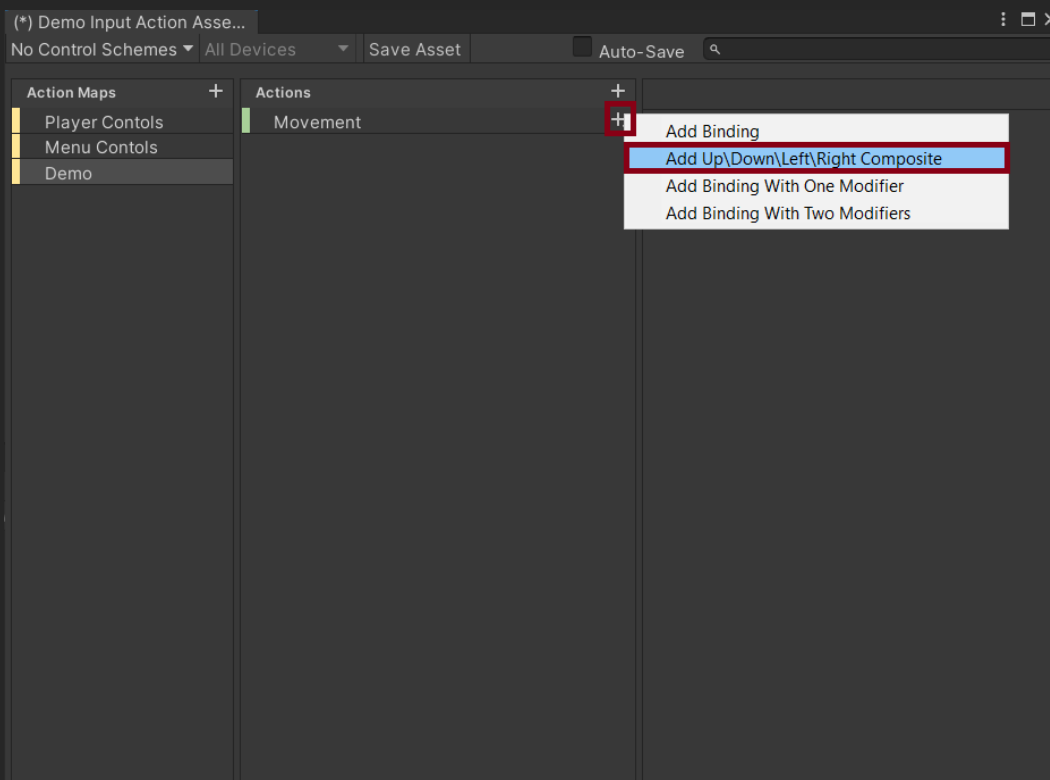
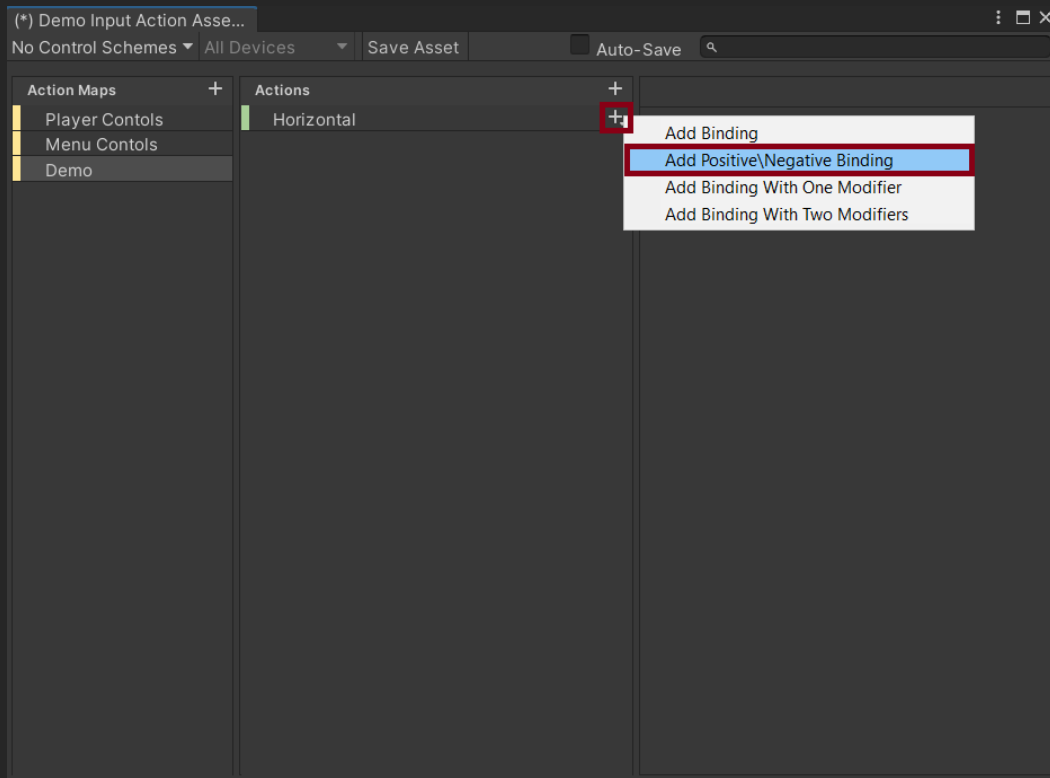


- What is a composite?

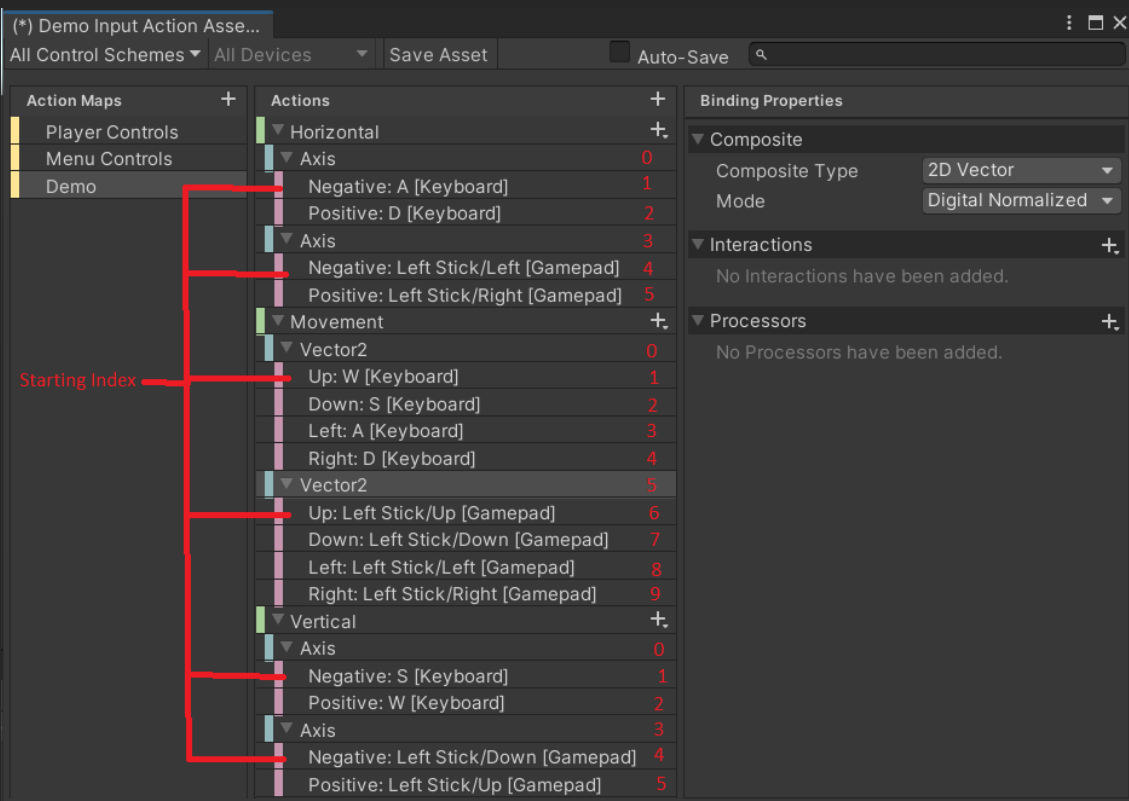
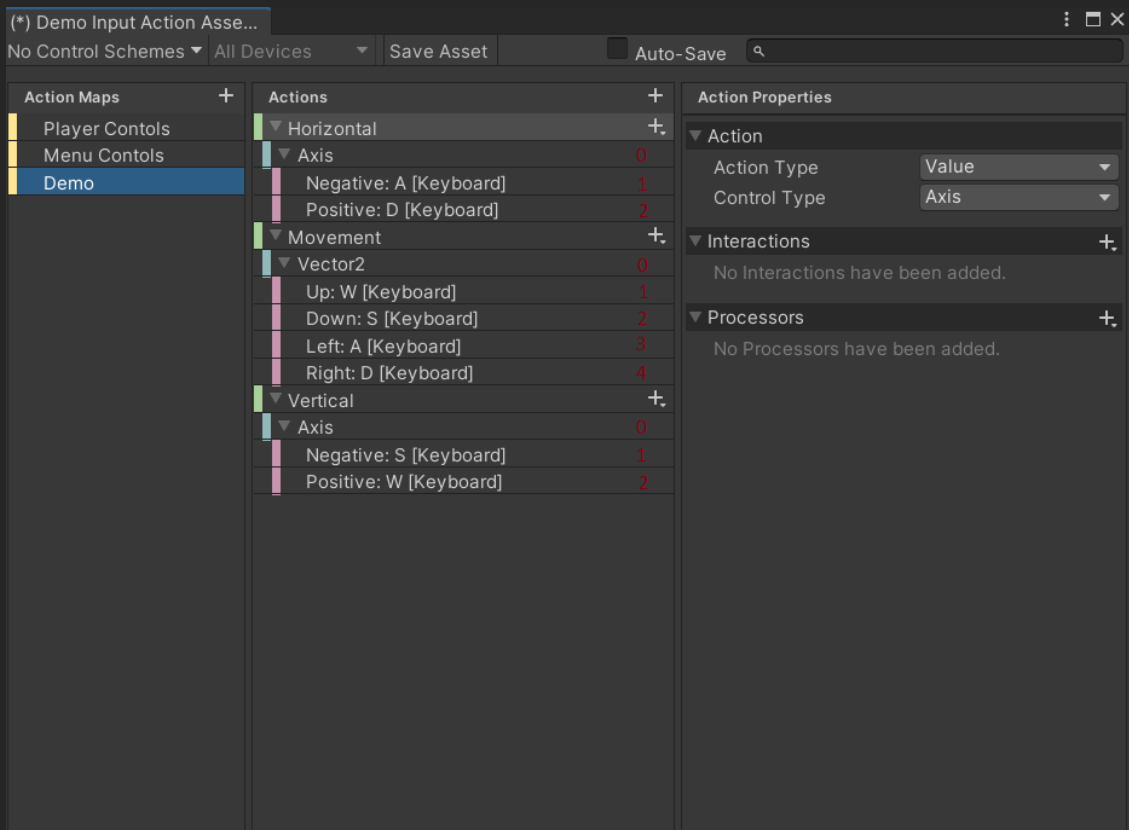


- How to create a composite?

Only available for Action Type = Value and Control Type = Axis/Vector 2/Vector 3.



- How are composites numbered?



- How are non-composites numbered?

